

Etapa 1: Diseño de esquemas flexibles en MongoDB

Caso de estudio: Ecommify

Autores: Daniel Porras, Oscar Clavijo, Camilo Porras

Fecha: junio 2026

1. Introducción

Ecommify es una plataforma de comercio electrónico híbrida diseñada bajo una arquitectura de microservicios y arquitectura orientada a eventos. De acuerdo con el documento técnico de diseño, la solución utiliza PostgreSQL para procesos críticos del negocio, como usuarios, órdenes, pagos e inventario, debido a sus capacidades ACID e integridad transaccional. Por otro lado, MongoDB se utiliza para componentes con alta flexibilidad estructural y alto volumen de datos, como catálogo extendido, comportamiento de usuario, recomendaciones, reseñas y logs de búsqueda.

En este contexto, MongoDB resulta adecuado para el catálogo de productos porque permite representar artículos con atributos variables según la categoría. Por ejemplo, un producto tecnológico puede tener memoria RAM, procesador y tarjeta gráfica, mientras que una prenda de vestir puede tener talla, color y material. Esta flexibilidad permite evolucionar el modelo sin modificar una estructura rígida de tablas.

El objetivo de este entregable es definir un esquema flexible para la colección *product_catalog*, documentar las decisiones de modelado, aplicar patrones de diseño documental y proponer consultas de validación sobre los datos.

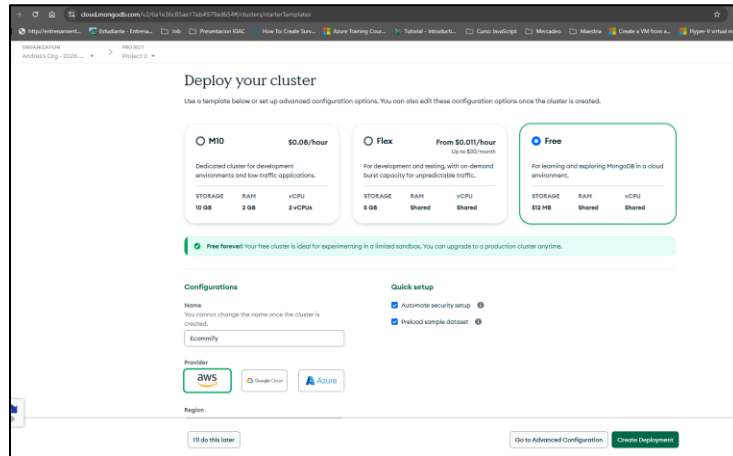
2. Configuración del entorno MongoDB Atlas

Para el desarrollo de la actividad se propone utilizar MongoDB Atlas con un cluster gratuito M0. Este ambiente permite crear una base de datos en la nube, conectarse desde Google Colab y validar el diseño documental mediante inserción y consulta de datos.

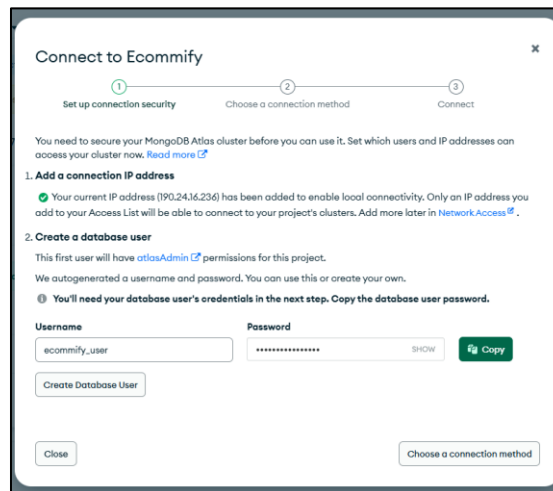
2.1 Pasos de configuración

Los pasos realizados o propuestos para configurar el ambiente son:

1. Crear una cuenta en MongoDB Atlas.
2. Crear un proyecto llamado Ecommify.
3. Crear un cluster gratuito M0.



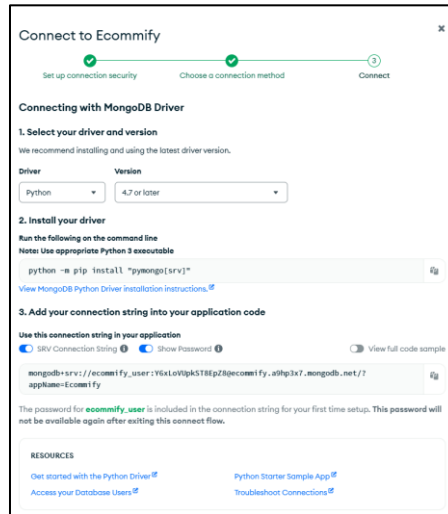
4. Configurar un usuario de base de datos desde la opción Database Access.
5. Asignar usuario y contraseña para la conexión.



6. Configurar el acceso de red desde Network Access.
7. Autorizar la dirección IP del equipo o permitir temporalmente acceso desde cualquier IP para pruebas académicas.



8. Obtener el connection string del cluster.



9. Abrir Google Colab.

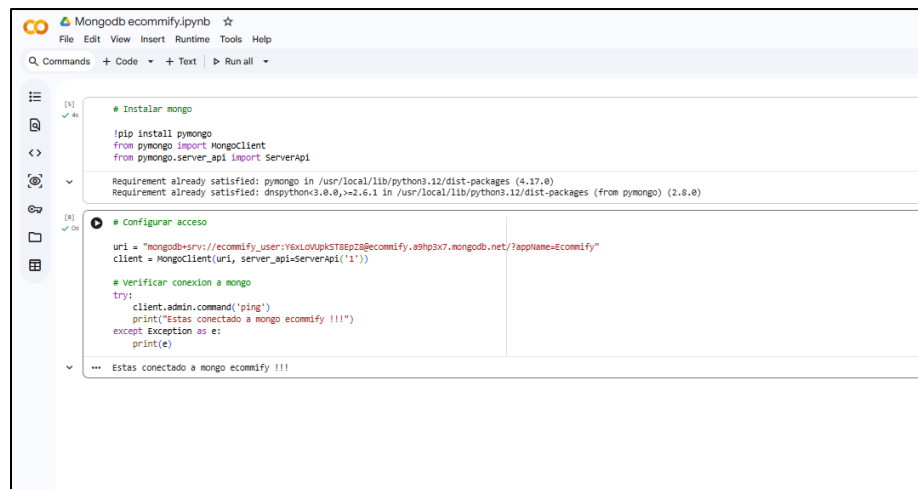
10. Instalar la librería pymongo.

11. Establecer conexión con el cluster.

12. Crear la base de datos ecommify_db.

13. Crear una colección de prueba.

14. Validar la conectividad insertando y consultando un documento.



2.2 Código de conexión desde Google Colab

Instalar mongo

```

!pip install pymongo
from pymongo import MongoClient
from pymongo.server_api import ServerApi

# Configurar acceso

uri =
"mongodb+srv://ecommify_user:Y6xLoVUpkST8EpZ8@ecommify.a9hp3x7.mongodb.
net/?appName=Ecommify"
client = MongoClient(uri, server_api=ServerApi('1'))

# Verificar conexion a mongo
try:
    client.admin.command('ping')
    print("Estas conectado a mongo ecommify !!!")
except Exception as e:
    print(e)

```

3. Contexto de MongoDB dentro de Ecommify

Según el documento técnico de diseño, MongoDB será utilizado para almacenar información semiestructurada y de alto volumen. Las colecciones principales propuestas para Ecommify son:

<i>Colección</i>	<i>Descripción</i>
<i>product_catalog</i>	Catálogo extendido de productos
<i>user_behavior</i>	Navegación y comportamiento de usuarios
<i>recommendations</i>	Recomendaciones generadas
<i>product_reviews</i>	Comentarios y reseñas de productos
<i>search_logs</i>	Historial de búsquedas

La colección central de este entregable será `product_catalog`, debido a que corresponde al catálogo flexible de productos. Esta colección debe soportar diferentes categorías, atributos variables, imágenes, etiquetas, calificaciones y métricas calculadas.

4. Diseño de esquema flexible para la colección `product_catalog`

La colección *`product_catalog`* representa el catálogo extendido de productos de Ecommify. Se diseña como una colección flexible, donde cada documento representa un producto y puede tener una estructura diferente dentro del campo `attributes`.

4.1 Atributos comunes

Aunque cada producto puede tener atributos específicos, se definen campos comunes que permiten mantener consistencia básica en el catálogo.

Campo	Tipo sugerido	Descripción
<code>_id</code>	String	Identificador único del producto
<code>name</code>	String	Nombre comercial del producto
<code>description</code>	String	Descripción del producto
<code>category</code>	String	Categoría principal
<code>price</code>	Number	Precio del producto
<code>currency</code>	String	Moneda del precio
<code>seller_id</code>	String	Identificador del vendedor
<code>seller_region</code>	String	Región del vendedor
<code>attributes</code>	Object	Atributos variables según categoría
<code>tags</code>	Array	Etiquetas para búsqueda
<code>images</code>	Array	Imágenes del producto
<code>ratings</code>	Object	Calificación promedio y cantidad de reseñas

metrics	Object	Métricas calculadas
recent_reviews	Array	Subconjunto de reseñas recientes
created_at	Date	Fecha de creación
updated_at	Date	Fecha de última actualización

El campo más importante para la flexibilidad es `attributes`, ya que permite almacenar características diferentes según el tipo de producto.

5. Ejemplos de documentos

5.1 Producto de categoría tecnología

```
{
  "_id": "PROD-1001",
  "name": "Laptop Gamer",
  "description": "Laptop de alto rendimiento para videojuegos, diseño y tareas
intensivas.",
  "category": "Tecnología",
  "price": 4500,
  "currency": "USD",
  "seller_id": "SELLER-001",
  "seller_region": "CO-BOG",
  "attributes": {
    "ram": "32GB",
    "cpu": "Intel i9",
    "gpu": "RTX 4080",
    "storage": "1TB SSD",
    "screen": "17 inches"
  },
  "tags": [
    "gaming",
    "high-performance",
    "laptop",
    "tecnologia"
  ],
}
```

```

"images": [
  "img_laptop_1.jpg",
  "img_laptop_2.jpg"
],
"ratings": {
  "average": 4.8,
  "count": 500
},
"metrics": {
  "total_units_sold": 1200,
  "views_count": 25000,
  "conversion_rate": 0.08
},
"recent_reviews": [
  {
    "user_id": 1001,
    "rating": 5,
    "comment": "Excelente rendimiento para videojuegos."
  },
  {
    "user_id": 1002,
    "rating": 4,
    "comment": "Buen equipo, aunque el precio es alto."
  }
],
"created_at": "2026-05-25T10:00:00",
"updated_at": "2026-05-25T10:00:00"
}

```

5.2 Producto de categoría ropa

```

{
  "_id": "PROD-2001",
  "name": "Chaqueta impermeable",
  "description": "Chaqueta resistente al agua para clima frío y actividades al aire libre.",
  "category": "Ropa",
  "price": 180,
  "currency": "USD",
  "seller_id": "SELLER-002",
  "seller_region": "CO-ANT",
  "attributes": {
    "size": "M",

```

```

    "color": "Negro",
    "material": "Poliéster",
    "gender": "Unisex"
  },
  "tags": [
    "chaqueta",
    "impermeable",
    "ropa",
    "outdoor"
  ],
  "images": [
    "img_jacket_1.jpg",
    "img_jacket_2.jpg"
  ],
  "ratings": {
    "average": 4.5,
    "count": 120
  },
  "metrics": {
    "total_units_sold": 340,
    "views_count": 8700,
    "conversion_rate": 0.05
  },
  "recent_reviews": [
    {
      "user_id": 1010,
      "rating": 5,
      "comment": "Muy buena calidad."
    }
  ],
  "created_at": "2026-05-25T10:00:00",
  "updated_at": "2026-05-25T10:00:00"
}

```

5.3 Producto de categoría libros

```

{
  "_id": "PROD-3001",
  "name": "Cien años de soledad",
  "description": "Novela literaria de Gabriel García Márquez.",
  "category": "Libros",
  "price": 25,

```



```
"currency": "USD",
"seller_id": "SELLER-003",
"seller_region": "CO-ATL",
"attributes": {
  "author": "Gabriel García Márquez",
  "language": "Español",
  "format": "Físico",
  "pages": 471
},
"tags": [
  "literatura",
  "novela",
  "libro",
  "clasico"
],
"images": [
  "img_book_1.jpg"
],
"ratings": {
  "average": 4.9,
  "count": 900
},
"metrics": {
  "total_units_sold": 5000,
  "views_count": 40000,
  "conversion_rate": 0.12
},
"recent_reviews": [
  {
    "user_id": 1020,
    "rating": 5,
    "comment": "Una obra fundamental de la literatura latinoamericana."
  }
],
"created_at": "2026-05-25T10:00:00",
"updated_at": "2026-05-25T10:00:00"
}
```

6. Decisión embedding vs referencing

Para el diseño de Ecommify se propone una estrategia mixta entre embedding y referencing.

6.1 Uso de embedding

Se recomienda usar embedding para información que se consulta frecuentemente junto con el producto y que no crece de forma ilimitada.

Elementos embebidos en product_catalog:

Elemento	Justificación
attributes	Hace flexible el producto según la categoría
tags	Permite búsquedas rápidas por palabras clave
images	Se consultan junto con el producto
ratings	Evita calcular promedio y conteo en cada consulta
metrics	Mejora rendimiento de consultas frecuentes
recent_reviews	Permite mostrar reseñas recientes sin consultar otra colección

El embedding mejora el rendimiento de lectura porque reduce la necesidad de hacer consultas adicionales para mostrar la ficha del producto.

6.2 Uso de referencing

Se recomienda usar referencing para entidades que pueden crecer demasiado, que pertenecen a otros dominios o que se actualizan de forma independiente.

Entidades referenciadas:

Entidad	Colección	Justificación
Reseñas completas	product_reviews	Un producto puede tener miles de reseñas
Vendedores	sellers	Un vendedor puede tener muchos productos

Usuarios	PostgreSQL / servicio de usuarios	Requiere consistencia e integridad
Historial de navegación	user_behavior	Alto volumen de eventos
Recomendaciones	recommendations	Se generan de forma analítica

Ejemplo de reseña referenciada:

```
{
  "_id": "REV-0001",
  "product_id": "PROD-1001",
  "user_id": 1001,
  "rating": 5,
  "comment": "Excelente laptop para videojuegos y trabajo pesado.",
  "verified_purchase": true,
  "created_at": "2026-05-25T12:00:00"
}
```

La relación se realiza mediante el campo ***product_id***, que apunta al identificador del producto en ***product_catalog***.

7. Patrones de modelado aplicados

7.1 Computed Pattern

El Computed Pattern consiste en almacenar valores calculados previamente para evitar recalcularlos en cada consulta. En Ecommify se aplica en los campos ratings y metrics.

Justificación:

El catálogo se consulta constantemente. Si cada vez que se muestra un producto se calcula el promedio de calificaciones o la cantidad de ventas desde otras colecciones, el rendimiento puede verse afectado. Por esta razón, se guardan métricas precalculadas dentro del documento del producto.

Ventajas:

- Reduce tiempo de respuesta.
- Disminuye carga de agregaciones.
- Mejora experiencia del usuario.
- Es útil para ordenar productos por ventas, calificación o popularidad.

Trade-off:

- Los valores precalculados deben actualizarse cuando cambian las reseñas, ventas o visualizaciones.

7.2 Subset Pattern

El Subset Pattern consiste en almacenar dentro de un documento solo una parte pequeña de una relación de alto volumen. En Ecommify se aplica en `recent_reviews`, donde se almacenan solo algunas reseñas recientes o destacadas.

Justificación:

Un producto puede tener miles de reseñas. Guardarlas todas dentro del producto haría crecer demasiado el documento. Por eso, se almacena solo un subconjunto en `product_catalog`, mientras que la colección `product_reviews` conserva el historial completo.

Ventajas:

- Mejora la consulta del detalle del producto.
- Evita crecimiento excesivo del documento.
- Permite mostrar información relevante rápidamente.

Trade-off:

- Para consultar todas las reseñas se requiere acceder a la colección `product_reviews`.

7.3 Approximation Pattern

El Approximation Pattern se utiliza cuando no es necesario tener precisión absoluta en tiempo real. En Ecommify se puede aplicar sobre métricas como:

- `views_count`
- `conversion_rate`
- `total_units_sold` para tableros analíticos
- Conteo aproximado de búsquedas
- Métricas de popularidad

Justificación:

Las visualizaciones de productos pueden ocurrir en grandes volúmenes. Actualizar el contador exacto en cada evento puede generar alta carga de escritura. Por eso, se pueden acumular eventos y actualizar el contador por lotes.

Ventajas:

- Reduce escrituras constantes.
- Mejora rendimiento.
- Es adecuado para analítica y popularidad.

Trade-off:

- Puede existir una pequeña diferencia entre el valor real y el valor almacenado.

7.4 Polymorphic Pattern

El Polymorphic Pattern permite almacenar documentos con estructuras diferentes dentro de una misma colección. En Ecommify aplica directamente a `product_catalog`, porque todos los productos se almacenan en la misma colección, pero sus atributos cambian según la categoría.

Ventajas:

- Permite flexibilidad estructural.
- Evita crear una colección por cada tipo de producto.
- Facilita la evolución del catálogo.

Trade-off:

- Se requiere controlar la calidad de datos mediante validaciones de aplicación o reglas de esquema.

7.5 Bucket Pattern

El Bucket Pattern aplica principalmente a la colección `user_behavior`, donde se almacenan eventos históricos de navegación.

Justificación:

El historial de navegación tiene un crecimiento extremadamente alto. Agrupar eventos por usuario y periodo permite manejar mejor el volumen, facilitar consultas analíticas y organizar los datos por ventanas de tiempo.

8. Implementación: generación e inserción de 1000 productos

El siguiente código permite generar 1000 documentos de ejemplo para la colección `product_catalog`.

```
# Insertar los 1000 registros

db = client["ecommyfy_db"]
product_catalog = db["product_catalog"]

product_catalog.delete_many({})

categories = ["Tecnología", "Ropa", "Hogar", "Deportes", "Libros"]
regions = ["CO-BOG", "CO-ANT", "CO-ATL", "CO-VAC", "CO-SAN"]

def generate_attributes(category):
    if category == "Tecnología":
        return {
            "ram": choice(["8GB", "16GB", "32GB"]),
            "cpu": choice(["Intel i5", "Intel i7", "Intel i9", "AMD Ryzen 7"]),
            "storage": choice(["256GB SSD", "512GB SSD", "1TB SSD"]),
            "gpu": choice(["Integrada", "RTX 3060", "RTX 4080"])
        }
    elif category == "Ropa":
        return {
            "size": choice(["S", "M", "L", "XL"]),
            "color": choice(["Negro", "Blanco", "Azul", "Rojo"]),
            "material": choice(["Algodón", "Poliéster", "Denim"])
        }
    elif category == "Hogar":
        return {
            "material": choice(["Madera", "Metal", "Plástico"]),
            "dimensions": choice(["Pequeño", "Mediano", "Grande"]),
            "brand": choice(["Homecenter", "Ikea", "Haceb"])
        }
    elif category == "Deportes":
        return {
            "sport_type": choice(["Running", "Fútbol", "Ciclismo", "Gimnasio"]),
            "size": choice(["S", "M", "L"]),
            "brand": choice(["Nike", "Adidas", "Puma"])
        }
    else:
        return {
            "author": choice(["Gabriel García Márquez", "Isabel Allende", "Mario Vargas Llosa"]),
            "language": choice(["Español", "Inglés"]),
            "format": choice(["Físico", "Digital"])
        }
```

```
[12]
✓ 3s
documents = []

for i in range(1, 1001):
    category = choice(categories)

    product = {
        "_id": f"PROD-{i:04d}",
        "name": f"{category} Producto {i}",
        "description": f"Descripción del producto {i} de la categoría {category}",
        "category": category,
        "price": randint(20, 5000),
        "currency": "USD",
        "seller_id": f"SELLER-{randint(1, 50):03d}",
        "seller_region": choice(regions),
        "attributes": generate_attributes(category),
        "tags": [category.lower(), "ecommify", "catalogo"],
        "images": [f"img_{i}_1.jpg", f"img_{i}_2.jpg"],
        "ratings": {
            "average": round(uniform(1, 5), 1),
            "count": randint(0, 1000)
        },
        "metrics": {
            "total_units_sold": randint(0, 5000),
            "views_count": randint(0, 100000),
            "conversion_rate": round(uniform(0.01, 0.20), 2)
        },
        "recent_reviews": [
            {
                "user_id": randint(1, 5000),
                "rating": randint(1, 5),
                "comment": "Comentario de ejemplo sobre el producto."
            }
        ],
        "created_at": datetime.now(),
        "updated_at": datetime.now()
    }

    documents.append(product)

product_catalog.insert_many(documents)

print("Total productos insertados:", product_catalog.count_documents({}))

... Total productos insertados: 1000
```

9. Creación de colección product_reviews

Además del catálogo, se crea una colección externa para las reseñas completas.

```
[13]
✓ 2s
# Generacion de reseñas

product_reviews = db["product_reviews"]
product_reviews.delete_many({})

reviews = []

for i in range(1, 501):
    review = {
        "_id": f"REV-{i:04d}",
        "product_id": f"PROD-{randint(1, 1000):04d}",
        "user_id": randint(1, 5000),
        "rating": randint(1, 5),
        "comment": "Reseña generada como dato de prueba para Ecommify.",
        "verified_purchase": choice([True, False]),
        "created_at": datetime.now()
    }
    reviews.append(review)

product_reviews.insert_many(reviews)

print("Total reseñas insertadas:", product_reviews.count_documents({}))

... Total reseñas insertadas: 500
```

10. Consultas de validación

Productos por categoría: Esta consulta valida que el esquema permite recuperar productos por categoría.

```
# Productos por categoría

for product in product_catalog.find({"category": "Tecnología"}).limit(5):
    print(product)

... {'_id': 'PROD-0008', 'name': 'Tecnología Producto 8', 'description': 'Descripción del producto 8 de la categoría Tecnología', 'category': 'Tecnología', 'price': 1275, 'cu...
{'_id': 'PROD-0009', 'name': 'Tecnología Producto 9', 'description': 'Descripción del producto 9 de la categoría Tecnología', 'category': 'Tecnología', 'price': 378, 'cu...
{'_id': 'PROD-0013', 'name': 'Tecnología Producto 13', 'description': 'Descripción del producto 13 de la categoría Tecnología', 'category': 'Tecnología', 'price': 4292, 'c...
{'_id': 'PROD-0019', 'name': 'Tecnología Producto 19', 'description': 'Descripción del producto 19 de la categoría Tecnología', 'category': 'Tecnología', 'price': 4648, 'c...
{'_id': 'PROD-0021', 'name': 'Tecnología Producto 21', 'description': 'Descripción del producto 21 de la categoría Tecnología', 'category': 'Tecnología', 'price': 426, 'c...
```

Consultar productos por precio: Esta consulta permite identificar productos cuyo precio es igual o superior a 1000 USD.

```
# Productos por precio

for product in product_catalog.find({"price": {"$gte": 1000}}).limit(5):
    print(product["_id"], product["name"], product["price"])

... PROD-0001 Deportes Producto 1 2116
PROD-0002 Libros Producto 2 1031
PROD-0003 Libros Producto 3 2787
PROD-0005 Hogar Producto 5 2769
PROD-0006 Hogar Producto 6 1156
```

Consultar productos por atributo específico: Esta consulta demuestra la flexibilidad de MongoDB para consultar atributos específicos dentro de un subdocumento dinámico.

```
# Consultar productos por atributo específico:

for product in product_catalog.find({"attributes.ram": "32GB"}).limit(5):
    print(product["_id"], product["name"], product["attributes"])

... PROD-0009 Tecnología Producto 9 {'ram': '32GB', 'cpu': 'Intel i5', 'storage': '1TB SSD', 'gpu': 'RTX 4080'}
PROD-0021 Tecnología Producto 21 {'ram': '32GB', 'cpu': 'Intel i5', 'storage': '1TB SSD', 'gpu': 'RTX 4080'}
PROD-0030 Tecnología Producto 30 {'ram': '32GB', 'cpu': 'Intel i9', 'storage': '1TB SSD', 'gpu': 'RTX 4080'}
PROD-0072 Tecnología Producto 72 {'ram': '32GB', 'cpu': 'Intel i5', 'storage': '256GB SSD', 'gpu': 'Integrada'}
PROD-0075 Tecnología Producto 75 {'ram': '32GB', 'cpu': 'Intel i5', 'storage': '512GB SSD', 'gpu': 'Integrada'}
```

Consultar productos por etiqueta: Esta consulta permite validar búsquedas por etiquetas.

```
# Consultar productos por etiqueta

for product in product_catalog.find({"tags": "tecnología"}).limit(5):
    print(product["_id"], product["name"], product["tags"])

... PROD-0008 Tecnología Producto 8 ['tecnología', 'ecommyfy', 'catalogo']
PROD-0009 Tecnología Producto 9 ['tecnología', 'ecommyfy', 'catalogo']
PROD-0013 Tecnología Producto 13 ['tecnología', 'ecommyfy', 'catalogo']
PROD-0019 Tecnología Producto 19 ['tecnología', 'ecommyfy', 'catalogo']
PROD-0021 Tecnología Producto 21 ['tecnología', 'ecommyfy', 'catalogo']
```


11. Aggregation pipelines

Promedio de precio por categoría: Este pipeline calcula el precio promedio de los productos agrupados por categoría.

```
# Promedio de precio por categoría

pipeline = [
    {
        "$group": {
            "_id": "$category",
            "average_price": {"$avg": "$price"},
            "total_products": {"$sum": 1}
        }
    },
    {
        "$sort": {"average_price": -1}
    }
]

for result in product_catalog.aggregate(pipeline):
    print(result)

... {'_id': 'Deportes', 'average_price': 2558.5219298245615, 'total_products': 228}
{'_id': 'Hogar', 'average_price': 2541.320224719101, 'total_products': 178}
{'_id': 'Libros', 'average_price': 2492.4766355140187, 'total_products': 214}
{'_id': 'Ropa', 'average_price': 2451.758823529412, 'total_products': 170}
{'_id': 'Tecnología', 'average_price': 2363.690476190476, 'total_products': 210}
```

Top 10 productos más vendidos: Esta consulta aprovecha el Computed Pattern porque usa el campo `metrics.total_units_sold`, que ya se encuentra precalculado.

```
# Top 10 productos más vendidos

for product in product_catalog.find().sort("metrics.total_units_sold", -1).limit(10):
    print(product["_id"], product["name"], product["metrics"]["total_units_sold"])

... PROD-0449 Libros Producto 449 4994
PROD-0926 Deportes Producto 926 4994
PROD-0748 Tecnología Producto 748 4987
PROD-0229 Libros Producto 229 4979
PROD-0920 Deportes Producto 920 4972
PROD-0628 Tecnología Producto 628 4971
PROD-0168 Hogar Producto 168 4971
PROD-0787 Hogar Producto 787 4967
PROD-0985 Deportes Producto 985 4966
PROD-0521 Libros Producto 521 4965
```

Productos con mejor calificación

Esta consulta muestra los productos mejor calificados, considerando únicamente aquellos con al menos 50 calificaciones.

```
[21]
✓ Os # Productos con mejor calificación

for product in product_catalog.find({"ratings.count": {"$gte": 50}}).sort("ratings.average", -1).limit(10):
    print(product["_id"], product["name"], product["ratings"]["average"])

*** PROD-0669 Libros Producto 669 5.0
PROD-0417 Deportes Producto 417 5.0
PROD-0792 Libros Producto 792 5.0
PROD-0129 Tecnología Producto 129 5.0
PROD-0289 Hogar Producto 289 5.0
PROD-0735 Deportes Producto 735 5.0
PROD-0945 Ropa Producto 945 5.0
PROD-0911 Deportes Producto 911 5.0
PROD-0340 Hogar Producto 340 5.0
PROD-0197 Deportes Producto 197 5.0
```

Cantidad de productos por región del vendedor: Este pipeline permite analizar la distribución de productos por región del vendedor.

```
[22]
✓ Os pipeline = [
    {
        "$group": {
            "_id": "$seller_region",
            "total_products": {"$sum": 1}
        }
    },
    {
        "$sort": {"total_products": -1}
    }
]

for result in product_catalog.aggregate(pipeline):
    print(result)

*** {'_id': 'CO-BOG', 'total_products': 239}
{'_id': 'CO-ANT', 'total_products': 207}
{'_id': 'CO-VAC', 'total_products': 201}
{'_id': 'CO-ATL', 'total_products': 187}
{'_id': 'CO-SAN', 'total_products': 166}
```

Índices recomendados

Para mejorar el rendimiento de las consultas frecuentes, se recomienda crear los siguientes índices:

```
# Creacion de indices

product_catalog.create_index({"category": 1})
product_catalog.create_index({"price": 1})
product_catalog.create_index({"tags": 1})
product_catalog.create_index({"seller_id": 1})
product_catalog.create_index({"seller_region": 1})
product_catalog.create_index({"ratings.average": -1})
product_catalog.create_index({"metrics.total_units_sold": -1})
product_catalog.create_index({"attributes.ram": 1})
product_reviews.create_index({"product_id": 1})
```

Justificación:

- category: mejora consultas por categoría.
- price: mejora filtros por precio.
- tags: mejora búsquedas por etiquetas.
- seller_id: permite consultar productos por vendedor.
- seller_region: permite análisis regional.
- ratings.average: permite ordenar por calificación.
- metrics.total_units_sold: permite ordenar por ventas.
- product_id en product_reviews: permite consultar reseñas de un producto específico.

12.Ventajas del diseño propuesto

El diseño propuesto ofrece las siguientes ventajas:

1. Permite manejar productos con atributos variables.
2. Reduce rigidez frente a cambios en el catálogo.
3. Mejora rendimiento de lectura mediante datos embebidos.
4. Evita crecimiento excesivo del documento mediante referencias.
5. Soporta consultas por categoría, precio, etiquetas, región y métricas.
6. Permite aplicar patrones documentales especializados.
7. Se alinea con la arquitectura híbrida de Ecommify.
8. Facilita la integración con recomendaciones y analítica.

13.Trade-offs del diseño

Aunque el diseño es flexible, también presenta algunos trade-offs:

Decisión	Beneficio	Riesgo	Mitigación
Usar attributes flexible	Permite categorías variables	Puede generar datos inconsistentes	Validar desde aplicación
Embeber ratings y metrics	Mejora lectura	Requiere actualización periódica	Actualización por eventos
Embeber recent_reviews	Mejora detalle de producto	No contiene todas las reseñas	Consultar product_reviews

Referenciar reseñas completas	Evita documentos grandes	Requiere consulta adicional	Índice por product_id
Usar métricas aproximadas	Reduce escrituras	Puede perder exactitud	Recalcular por lotes

14. Conclusiones

MongoDB es una opción adecuada para el catálogo extendido de Ecommify porque permite trabajar con documentos flexibles y atributos variables por categoría. Esta característica es fundamental en una plataforma de comercio electrónico, donde los productos no siempre comparten las mismas propiedades.

El diseño de la colección product_catalog permite combinar campos comunes con un subdocumento flexible llamado attributes. Esto facilita representar productos tecnológicos, ropa, artículos del hogar, productos deportivos y libros dentro de una misma colección.

La estrategia de modelado combina embedding y referencing. Se embebe la información consultada frecuentemente, como atributos, etiquetas, imágenes, calificaciones y métricas calculadas. Al mismo tiempo, se referencian entidades de alto crecimiento, como reseñas completas y eventos de comportamiento de usuario.

Los patrones Computed Pattern, Subset Pattern, Approximation Pattern, Polymorphic Pattern y Bucket Pattern permiten optimizar el rendimiento, controlar el crecimiento de documentos y soportar datos heterogéneos.

Finalmente, las consultas y pipelines propuestos validan que el esquema soporta requisitos funcionales del catálogo, como búsqueda por categoría, filtros por precio, consulta de atributos variables, ranking por ventas, ranking por calificación y análisis por región del vendedor.

15. Referencias

- MongoDB, Inc. Data Modeling Patterns. MongoDB Manual.
- MongoDB, Inc. Sharding. MongoDB Manual.
- Documento Técnico de Diseño – Ecommify.